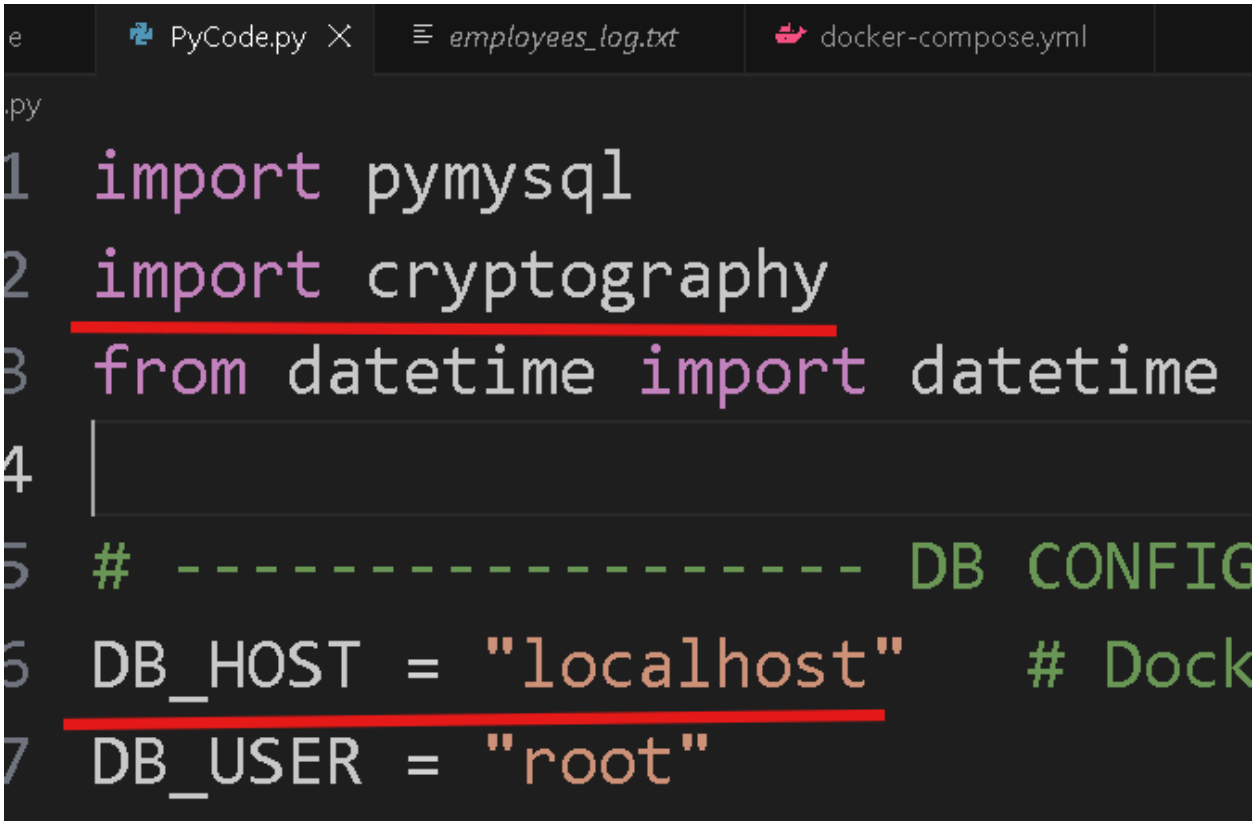
Docker Compose with Volumes

Lets achieve mount bind, using docker compose.
In mount bind, file within the container which is linked with file that is in physical machine.



Python code:

```
import pymysql
import cryptography
from datetime import datetime

# ------------------ DB CONFIG ------------------ #
DB_HOST = "localhost"   # Docker service name
DB_USER = "root"
DB_PASSWORD = "ghat123@gmail.com"
DB_NAME = "employee_db"

# ------------------ LOG CONFIG ------------------ #
LOG_FILE = "employees_log.txt"

def log_to_file(action, data):
    try:
        with open(LOG_FILE, "a") as f:
```

```
            f.write(f"{'-'*50}\n")
            f.write(f"Timestamp : {datetime.now()}\n")
            f.write(f"Action    : {action}\n")
            f.write(f"Data      : {data}\n")
    except Exception as e:
        print("❌ File logging error:", e)

# ------------------ DB CONNECTION ------------------ #
def connect_db():
    return pymysql.connect(
        host=DB_HOST,
        user=DB_USER,
        password=DB_PASSWORD,
        cursorclass=pymysql.cursors.Cursor
    )

def setup_database():
    conn = connect_db()
    cursor = conn.cursor()

    cursor.execute(f"CREATE DATABASE IF NOT EXISTS {DB_NAME}")
    cursor.execute(f"USE {DB_NAME}")

    cursor.execute("""
        CREATE TABLE IF NOT EXISTS employees (
            id INT AUTO_INCREMENT PRIMARY KEY,
            name VARCHAR(100) NOT NULL,
            email VARCHAR(100),
            phone VARCHAR(15) NOT NULL,
            salary FLOAT,
            aptitude_score FLOAT NOT NULL
        )
    """)

    conn.commit()
    return conn, cursor

# ------------------ ADD EMPLOYEE ------------------ #
def add_employee(cursor, conn):
    print("\n--- Add Employee ---")

    name = input("Enter Name (required): ").strip()
    phone = input("Enter Phone (required): ").strip()
    email = input("Enter Email (optional): ").strip()
```

```
    salary = input("Enter Salary (optional): ").strip()
    aptitude = input("Enter Aptitude Score (0-10) (required): ").strip()

    if not name or not phone or not aptitude:
        print("❌ Employee NOT added. Required fields missing.\n")
        return

    try:
        salary = float(salary) if salary else None
        aptitude = float(aptitude)

        if aptitude < 0 or aptitude > 10:
            print("❌ Aptitude score must be between 0 and 10.\n")
            return

        cursor.execute("""
            INSERT INTO employees (name, email, phone, salary, aptitude_score)
            VALUES (%s, %s, %s, %s, %s)
        """, (name, email if email else None, phone, salary, aptitude))

        conn.commit()

        # 🔥 Log to file
        log_to_file("ADD", {
            "name": name,
            "email": email,
            "phone": phone,
            "salary": salary,
            "aptitude": aptitude
        })

        print("✅ Employee added successfully!\n")

    except Exception as e:
        print("❌ Error:", e)

# ------------------ UPDATE EMPLOYEE ------------------ #
def update_employee(cursor, conn):
    print("\n--- Update Employee ---")
    emp_id = input("Enter Employee ID: ").strip()

    if not emp_id:
        print("❌ ID not provided.\n")
        return
```

```
    try:
        cursor.execute("SELECT * FROM employees WHERE id=%s", (emp_id,))
        if not cursor.fetchone():
            print("❌ Employee not found.\n")
            return

        print("""
a. Update Name
b. Update Email
c. Update Phone
d. Update Salary
e. Update Aptitude Score
""")

        choice = input("Choose option: ").lower()

        field_map = {
            'a': 'name',
            'b': 'email',
            'c': 'phone',
            'd': 'salary',
            'e': 'aptitude_score'
        }

        if choice not in field_map:
            print("❌ Invalid choice.\n")
            return

        new_value = input("Enter new value: ").strip()

        if not new_value:
            print("❌ Value cannot be empty.\n")
            return

        if field_map[choice] == 'salary':
            new_value = float(new_value)
        elif field_map[choice] == 'aptitude_score':
            new_value = float(new_value)
            if new_value < 0 or new_value > 10:
                print("❌ Aptitude must be between 0 and 10.\n")
                return

        query = f"UPDATE employees SET {field_map[choice]}=%s WHERE id=%s"
```

```
        cursor.execute(query, (new_value, emp_id))
        conn.commit()

        # 🔥 Log to file
        log_to_file("UPDATE", {
            "id": emp_id,
            "field": field_map[choice],
            "new_value": new_value
        })

        print("✅ Employee updated successfully!\n")

    except Exception as e:
        print("❌ Error:", e)

# ------------------ VIEW EMPLOYEES ------------------ #
def view_employees(cursor):
    print("\n--- View Employees ---")
    choice = input("Show all? (y/n): ").lower()

    try:
        if choice == 'y':
            cursor.execute("SELECT * FROM employees")
        else:
            ids = input("Enter min_id max_id: ").split()
            if len(ids) != 2:
                print("❌ Invalid input.\n")
                return

            cursor.execute(
                "SELECT * FROM employees WHERE id BETWEEN %s AND %s",
                (ids[0], ids[1])
            )

        rows = cursor.fetchall()

        if not rows:
            print("⚠️ No employees found.\n")
            return

        print("\n--- Employee Records ---")
        for row in rows:
            print(row)
        print()
```

```
    except Exception as e:
        print("❌ Error:", e)

# ------------------ DELETE EMPLOYEE ------------------ #
def delete_employee(cursor, conn):
    print("\n--- Delete Employee ---")
    emp_id = input("Enter Employee ID to delete: ").strip()

    if not emp_id:
        print("❌ ID not provided.\n")
        return

    try:
        cursor.execute("DELETE FROM employees WHERE id=%s", (emp_id,))
        conn.commit()

        if cursor.rowcount == 0:
            print("❌ Employee not found.\n")
        else:
            # 🔥 Log to file
            log_to_file("DELETE", {"id": emp_id})
            print("✅ Employee deleted successfully!\n")

    except Exception as e:
        print("❌ Error:", e)

# ------------------ MAIN LOOP ------------------ #
def main():
    try:
        conn, cursor = setup_database()
        print("✅ Database connected successfully!\n")

        while True:
            print("""
===== Employee Management =====
1. Add Employee
2. Update Employee
3. Show Employees
4. Delete Employee
5. Quit
""")

            choice = input("Enter choice (1-5): ").strip()
```

```
        if choice == '1':
            add_employee(cursor, conn)
        elif choice == '2':
            update_employee(cursor, conn)
        elif choice == '3':
            view_employees(cursor)
        elif choice == '4':
            delete_employee(cursor, conn)
        elif choice == '5':
            print("👋 Exiting program...")
            break
        else:
            print("❌ Invalid input.\n")

    cursor.close()
    conn.close()

except Exception as e:
    print("❌ Database connection failed:", e)

# ------------------ RUN ------------------ #
if __name__ == "__main__":
    main()
```

In terminal:

```
C:\Users\kiran\OneDrive\Desktop\Python Code>python PyCode.py
✅ Database connected successfully!


===== Employee Management =====
1. Add Employee
2. Update Employee
3. Show Employees
4. Delete Employee
5. Quit

Enter choice (1-5): 1

--- Add Employee ---
Enter Name (required): name1f name2l
Enter Phone (required): 34123
Enter Email (optional): 234@df.com
Enter Salary (optional): 234234
Enter Aptitude Score (0-10) (required): 5.2342
✅ Employee added successfully!


===== Employee Management =====
1. Add Employee
2. Update Employee
3. Show Employees
4. Delete Employee
5. Quit
```

In terminal if you chage. It has a functionallty to save logs in the text file.

employees_log.txt

```
----------------------------------------------
Timestamp : 2026-03-24 19:43:48.038088
Action    : ADD
Data      : {'name': 'name1f name2l', 'email': '234@df.com', 'phor
----------------------------------------------
Timestamp : 2026-03-24 19:44:38.410744
Action    : UPDATE
Data      : {'id': '8', 'field': 'name', 'new_value': 'newname'}
```

```
FROM python

WORKDIR /app
```

Employee logs txt file, will be created in /app. Need to bind `employees_log.txt` with the same text file within the app workdir. Path of this text file is present where docker compose exists. You can also provide related path.

```
services:
  mypythonapp:
    networks:
      - my-network
    volumes:
      - ./employees_log.txt:/app/employees_log.txt
    depends_on:
```

Python code:

```
import pymysql
# import cryptography
from datetime import datetime

# ----------------- DB CONFIG ----------------- #
```

```
DB_HOST = "mysqldb"   # Docker service name
DB_USER = "root"
DB_PASSWORD = "ghat123@gmail.com"
DB_NAME = "employee_db"

# ------------------ LOG CONFIG ------------------ #
LOG_FILE = "employees_log.txt"

def log_to_file(action, data):
    try:
        with open(LOG_FILE, "a") as f:
            f.write(f"{'-'*50}\n")
            f.write(f"Timestamp : {datetime.now()}\n")
            f.write(f"Action    : {action}\n")
            f.write(f"Data      : {data}\n")
    except Exception as e:
        print("❌ File logging error:", e)

# ------------------ DB CONNECTION ------------------ #
def connect_db():
    return pymysql.connect(
        host=DB_HOST,
        user=DB_USER,
        password=DB_PASSWORD,
        cursorclass=pymysql.cursors.Cursor
    )

def setup_database():
    conn = connect_db()
    cursor = conn.cursor()

    cursor.execute(f"CREATE DATABASE IF NOT EXISTS {DB_NAME}")
    cursor.execute(f"USE {DB_NAME}")

    cursor.execute("""
        CREATE TABLE IF NOT EXISTS employees (
            id INT AUTO_INCREMENT PRIMARY KEY,
            name VARCHAR(100) NOT NULL,
            email VARCHAR(100),
            phone VARCHAR(15) NOT NULL,
            salary FLOAT,
            aptitude_score FLOAT NOT NULL
        )
    """)
```

```
    conn.commit()
    return conn, cursor

# ------------------ ADD EMPLOYEE ------------------ #
def add_employee(cursor, conn):
    print("\n--- Add Employee ---")

    name = input("Enter Name (required): ").strip()
    phone = input("Enter Phone (required): ").strip()
    email = input("Enter Email (optional): ").strip()
    salary = input("Enter Salary (optional): ").strip()
    aptitude = input("Enter Aptitude Score (0-10) (required): ").strip()

    if not name or not phone or not aptitude:
        print("❌ Employee NOT added. Required fields missing.\n")
        return

    try:
        salary = float(salary) if salary else None
        aptitude = float(aptitude)

        if aptitude < 0 or aptitude > 10:
            print("❌ Aptitude score must be between 0 and 10.\n")
            return

        cursor.execute("""
            INSERT INTO employees (name, email, phone, salary, aptitude_score)
            VALUES (%s, %s, %s, %s, %s)
        """, (name, email if email else None, phone, salary, aptitude))

        conn.commit()

        # 🔥 Log to file
        log_to_file("ADD", {
            "name": name,
            "email": email,
            "phone": phone,
            "salary": salary,
            "aptitude": aptitude
        })

        print("✅ Employee added successfully!\n")
```

```
    except Exception as e:
        print("❌ Error:", e)

# ------------------ UPDATE EMPLOYEE ------------------ #
def update_employee(cursor, conn):
    print("\n--- Update Employee ---")
    emp_id = input("Enter Employee ID: ").strip()

    if not emp_id:
        print("❌ ID not provided.\n")
        return

    try:
        cursor.execute("SELECT * FROM employees WHERE id=%s", (emp_id,))
        if not cursor.fetchone():
            print("❌ Employee not found.\n")
            return

        print("""
a. Update Name
b. Update Email
c. Update Phone
d. Update Salary
e. Update Aptitude Score
""")

        choice = input("Choose option: ").lower()

        field_map = {
            'a': 'name',
            'b': 'email',
            'c': 'phone',
            'd': 'salary',
            'e': 'aptitude_score'
        }

        if choice not in field_map:
            print("❌ Invalid choice.\n")
            return

        new_value = input("Enter new value: ").strip()

        if not new_value:
            print("❌ Value cannot be empty.\n")
```

```
            return

        if field_map[choice] == 'salary':
            new_value = float(new_value)
        elif field_map[choice] == 'aptitude_score':
            new_value = float(new_value)
            if new_value < 0 or new_value > 10:
                print("❌ Aptitude must be between 0 and 10.\n")
                return

        query = f"UPDATE employees SET {field_map[choice]}=%s WHERE id=%s"
        cursor.execute(query, (new_value, emp_id))
        conn.commit()

        # 🔥 Log to file
        log_to_file("UPDATE", {
            "id": emp_id,
            "field": field_map[choice],
            "new_value": new_value
        })

        print("✅ Employee updated successfully!\n")

    except Exception as e:
        print("❌ Error:", e)

# ------------------ VIEW EMPLOYEES ------------------ #
def view_employees(cursor):
    print("\n--- View Employees ---")
    choice = input("Show all? (y/n): ").lower()

    try:
        if choice == 'y':
            cursor.execute("SELECT * FROM employees")
        else:
            ids = input("Enter min_id max_id: ").split()
            if len(ids) != 2:
                print("❌ Invalid input.\n")
                return

            cursor.execute(
                "SELECT * FROM employees WHERE id BETWEEN %s AND %s",
                (ids[0], ids[1])
            )
```

```
        rows = cursor.fetchall()

        if not rows:
            print("⚠️ No employees found.\n")
            return

        print("\n--- Employee Records ---")
        for row in rows:
            print(row)
        print()

    except Exception as e:
        print("❌ Error:", e)

# ------------------ DELETE EMPLOYEE ------------------ #
def delete_employee(cursor, conn):
    print("\n--- Delete Employee ---")
    emp_id = input("Enter Employee ID to delete: ").strip()

    if not emp_id:
        print("❌ ID not provided.\n")
        return

    try:
        cursor.execute("DELETE FROM employees WHERE id=%s", (emp_id,))
        conn.commit()

        if cursor.rowcount == 0:
            print("❌ Employee not found.\n")
        else:
            # 🔥 Log to file
            log_to_file("DELETE", {"id": emp_id})
            print("✅ Employee deleted successfully!\n")

    except Exception as e:
        print("❌ Error:", e)

# ------------------ MAIN LOOP ------------------ #
def main():
    try:
        conn, cursor = setup_database()
        print("✅ Database connected successfully!\n")
```

```
        while True:
            print("""
===== Employee Management =====
1. Add Employee
2. Update Employee
3. Show Employees
4. Delete Employee
5. Quit
""")

            choice = input("Enter choice (1-5): ").strip()

            if choice == '1':
                add_employee(cursor, conn)
            elif choice == '2':
                update_employee(cursor, conn)
            elif choice == '3':
                view_employees(cursor)
            elif choice == '4':
                delete_employee(cursor, conn)
            elif choice == '5':
                print("👋 Exiting program...")
                break
            else:
                print("❌ Invalid input.\n")

        cursor.close()
        conn.close()

    except Exception as e:
        print("❌ Database connection failed:", e)

# ------------------ RUN ------------------ #
if __name__ == "__main__":
    main()
```

```
PS C:\Users\kiran\OneDrive\Desktop\Python Code> docker-compose run mypythonapp
[+]  2/2t 2/22
 ✓ Network pythoncode_my-network Created
 ✓ Container mysqldb_new         Started
Container mysqldb_new Waiting
Container mysqldb_new Healthy
Container pythoncode-mypythonapp-run-563deeb50976 Creating
Container pythoncode-mypythonapp-run-563deeb50976 Created
✅ Database connected successfully!


===== Employee Management =====
1. Add Employee
2. Update Employee
3. Show Employees
4. Delete Employee
5. Quit

Enter choice (1-5): 
```

Quick Overview of Docker-Compose:

```
Docker Compose

A config file to make your task of managing and running containers easy.

Get rid of repetitive commands

All the services inside the config file, share same network

commands:
docker-compose up/down
-d detach mode
-v to remove networks/volumes upon stop
--build to again build image and run
```